A Project Report On

Data Analytics Internship Tasks

By

Shubham Girishbhai Sadadiya (23CS407028)

In Fulfillment Of

B.Sc. IT Sem. V (A.Y. 2025-2026)

Data Analytics Internship Tasks - Project Report

ABSTRACT

This project report encompasses the work completed during the Data Analytics Internship, focusing on three major analytical tasks: Superstore Sales Analysis, RavenStack SaaS Dataset Analysis, and E-Commerce Funnel Analysis. The projects involve end-to-end data processing, from raw data extraction and cleaning using Python and Pandas, to advanced data modeling and final presentation through interactive reporting layers like Excel and Power BI dashboards.

By analyzing transactional data, synthetic event-driven SaaS data, and user journey logs, the projects deliver actionable business intelligence, customer churn predictions, and conversion bottleneck identifications. The insights generated assist stakeholders in performance tracking, regional profitability analysis, and optimizing sales funnels.

Data Analytics Internship Tasks - Project Report

Chapter 1: Introduction

1.1 Project Summary

The internship tasks are divided into three core projects:

1. Superstore Sales Analysis & Dashboard: Transforms raw transactional data into a clean dataset enriched with KPIs like Profit Margin, culminating in a stakeholder-ready Excel Dashboard.

2. RavenStack Synthetic SaaS Dataset: Investigates a multi-table, event-driven dataset to discover what drives conversions, support load, and churn patterns for a stealth-mode SaaS startup.

3. Funnel Analysis - E-Commerce User Journey: Analyzes user journeys to identify conversion bottlenecks and optimize the sales funnel, presented through a Power BI dashboard.

1.2 Purpose

The purpose of these projects is to extract actionable business intelligence from raw data. By establishing robust ETL (Extract, Transform, Load) pipelines and interactive dashboards, the projects aim to help stakeholders make informed, data-driven decisions regarding sales, customer retention, and user experience.

1.3 Scope

The scope encompasses data extraction, cleaning (noise reduction, standardization), feature engineering (calculating KPIs like Profit Margin and Bounce Rate), relational data modeling (handling primary/foreign keys in SaaS data), and data visualization (Excel and Power BI).

1.4 Technology Review

* Python: Used for writing robust ETL scripts and data cleaning pipelines.

* Pandas & NumPy: Python libraries used extensively for data manipulation, aggregation, and

statistical modeling.

* Microsoft Excel: Used as the presentation layer for the Superstore Executive Dashboard.

* Power BI: Used for interactive, deep-dive visualizations in the E-Commerce Funnel Analysis.

Chapter 2: Project Management

2.1 Project Planning and Scheduling

The projects follow an Iterative Waterfall Model adapted for data analytics.

1. Requirement Phase: Understanding the data schemas and business questions.

2. Development Phase: Writing ETL scripts (Cleaning.py, Profite_Margin.py).

3. Testing Phase: Validating data integrity, checking for orphans in relational tables, and ensuring accurate KPI calculations.

4. Reporting Phase: Designing interactive dashboards and charts.

2.2 Risk Management

* Data Integrity Risks: Handling null fields, duplicate referrals, and noise in raw data. Mitigated by strict ETL validation rules.

* Performance Risks: Slowness in dashboard rendering with large datasets (e.g., 25,000 feature usage rows). Mitigated by pre-aggregating data in Python before loading into visualization tools.

2.3 Estimation

* Effort Estimation: Requires strong data wrangling skills in Python and logical modeling capabilities for relational data, along with UI/UX considerations for dashboard design.

* Cost Analysis: Utilized open-source libraries (Python, Pandas) and standard enterprise tools (Power BI, Excel), keeping direct software costs negligible.

Chapter 3: System Requirement Study

3.1 User Characteristics

* Data Analysts/Engineers: Responsible for maintaining the Python ETL scripts and updating the data source files.

* Business Stakeholders/Management: End-users who consume the Excel and Power BI dashboards to make strategic decisions.

3.2 Hardware and Software Requirements

* Software: Python 3.8+, Pandas Library, Microsoft Excel (2016+), Power BI Desktop.

* Hardware: Minimum Intel Core i3 Processor, 4.00 GB RAM, 64-bit Operating System.

3.3 Constraints

All relational data (like in RavenStack) must maintain referential integrity (valid foreign key constraints between accounts, subscriptions, and feature_usage). Data pipelines must handle missing values appropriately without skewing KPIs.

Chapter 4: System Analysis

4.1 Study of Current System

Prior to the dashboards, data existed in raw, disjointed CSV files. Stakeholders had to manually sift through thousands of transaction logs and event records to gauge performance, which was highly inefficient and prone to human error.

4.2 Problem and Weaknesses of Current System

* Lack of centralized Key Performance Indicators (KPIs).
* Inability to track multi-table relationships (e.g., linking support tickets directly to churn events).
* No visual tracking for funnel drop-offs or regional profit distributions.

4.3 Requirement of New System

A need for automated data pipelines that clean data, engineer necessary features, and output to structured formats that plug directly into visual dashboards for real-time, interactive exploration.

4.4 Feasibility Study

Given the availability of Python and standard visualization tools, building automated pipelines and dashboards is highly technically and economically feasible.

Chapter 5: System Design

5.1 Database Design / Data Structure Design

RavenStack Schema Example:

* accounts.csv: PK account_id, industry, plan_tier.

* subscriptions.csv: PK subscription_id, FK account_id, mrr_amount.

* feature_usage.csv: FK subscription_id, usage_duration_secs.

* churn_events.csv: FK account_id, reason_code.

Funnel Analysis Schema Example:

* funnel_analysis_data.csv: User_ID, Session_ID, Event (Browse, Add to Cart, Purchase), Revenue, Bounce_Flag.

5.2 System Procedural Design

Data Pipeline (Superstore Example):

1. Phase 1 (Data Cleaning): Convert to snake_case, cast datetime objects, remove summary rows.

2. Phase 2 (Feature Engineering): Calculate Profit Margin % = (Profit / Sales) * 100.

3. Phase 3 (Reporting): Export to Superstore_Data.xlsx and connect to the dashboard layer.

5.3 Input/Output and Interface Design

* Superstore Dashboard: Features Total Sales & Profit trends, regional performance hotspots, and category-wise profit distribution.

* Funnel Dashboard: Features step-by-step drop-off analysis, revenue insights by channel, and demographic breakdowns.

Chapter 6: Implementation Planning and Details

6.1 Implementation Environment

Development occurred locally utilizing Python virtual environments for script execution and Power BI Desktop / Excel for dashboard authoring. Code and data are separated into specific directories (Assets/, Dashboard/, Data/, Notebooks/).

6.2 Program/Modules Specification

* Cleaning.py: Handles data standardization and noise reduction.

* Profite_Margin.py: Handles feature engineering and KPI aggregations.

* Funnel_Aanalysis.pbix: Acts as the interactive reporting presentation layer.

6.3 Security Features

The datasets utilized (like RavenStack) are fully synthetic containing no Personally Identifiable Information (PII), ensuring complete compliance with data privacy standards during analysis and portfolio presentation.

6.4 Coding Standards

Python scripts adhere to PEP 8 standards, utilizing modular functions, descriptive variable names, and extensive comments/documentation to ensure reproducibility.

Chapter 7: Testing

7.1 Testing Plan

Testing focuses on Data Quality Assurance. The plan involves verifying data transformations, ensuring no data loss occurs during joins, and confirming dashboard calculations match raw aggregations.

7.2 Testing Strategy and Methods

* Pipeline Testing: Running Cleaning.py and verifying that specific noise rows (like "Grand Total") are completely removed.
* Referential Integrity Testing: Ensuring all subscription_ids in the feature usage table link cleanly back to valid subscriptions in the RavenStack dataset.
* KPI Validation: Manually calculating Profit Margin for a subset of rows to ensure the Profite_Margin.py script outputs the correct percentages.

Data Analytics Internship Tasks - Project Report

Chapter 8: Screenshots and User Manuals

Execution Steps:

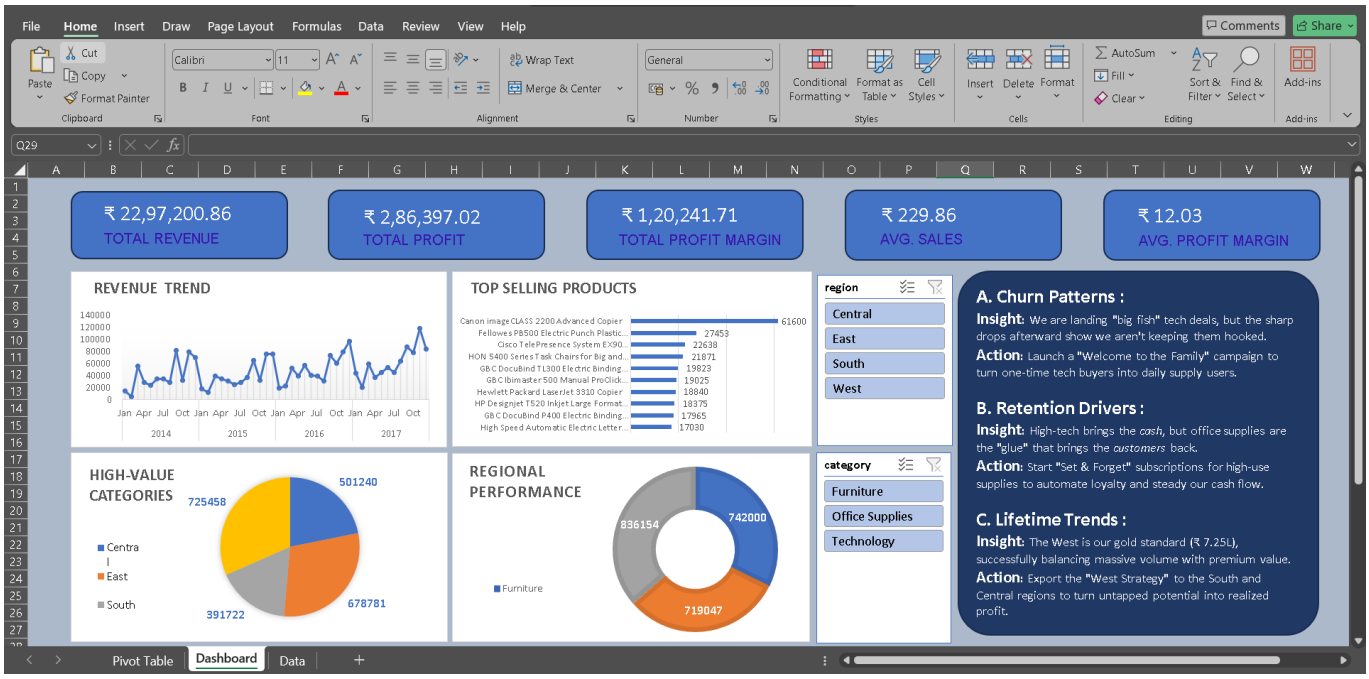
1. Navigate to the Notebooks folder.

2. Run python Cleaning.py

3. Run python Profite_Margin.py

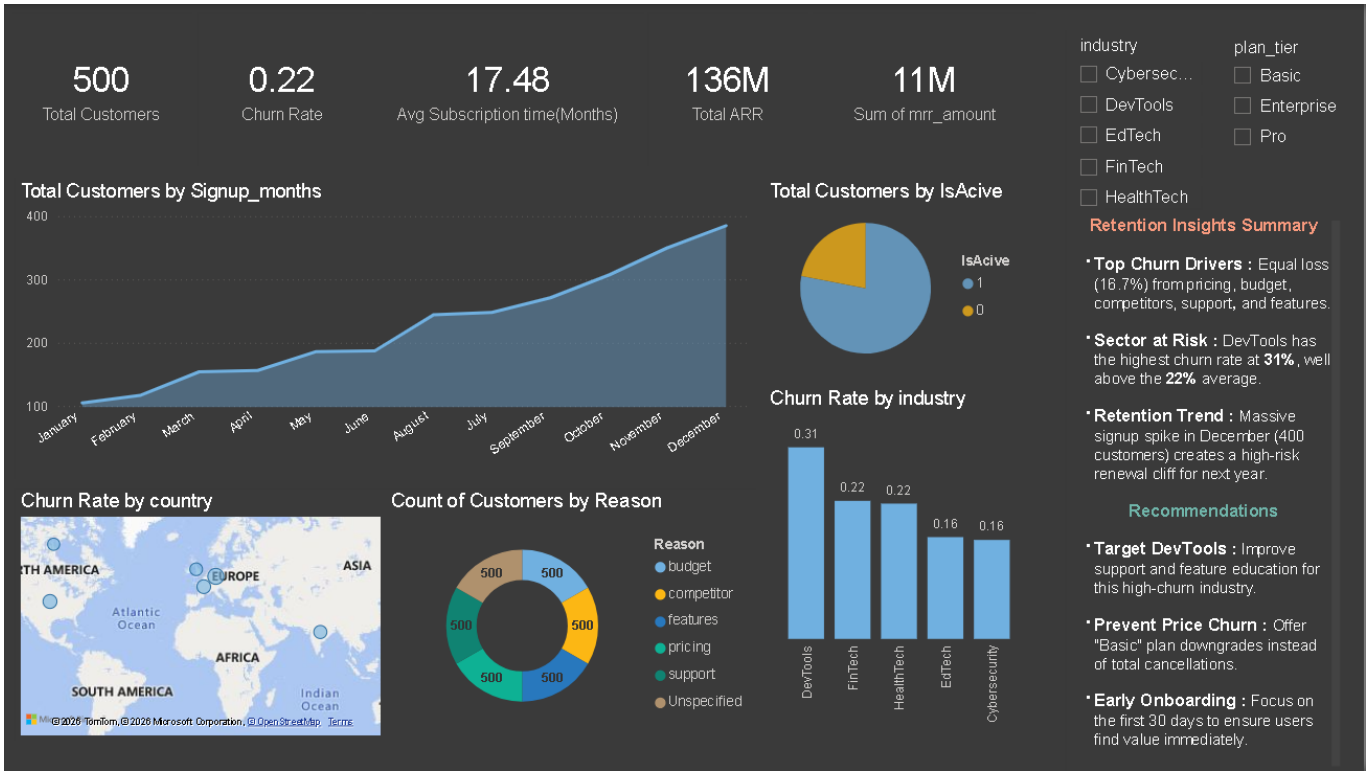
4. Open the respective .xlsx or .pbix dashboard file.

8.1 Superstore Sales Analysis & Dashboard:

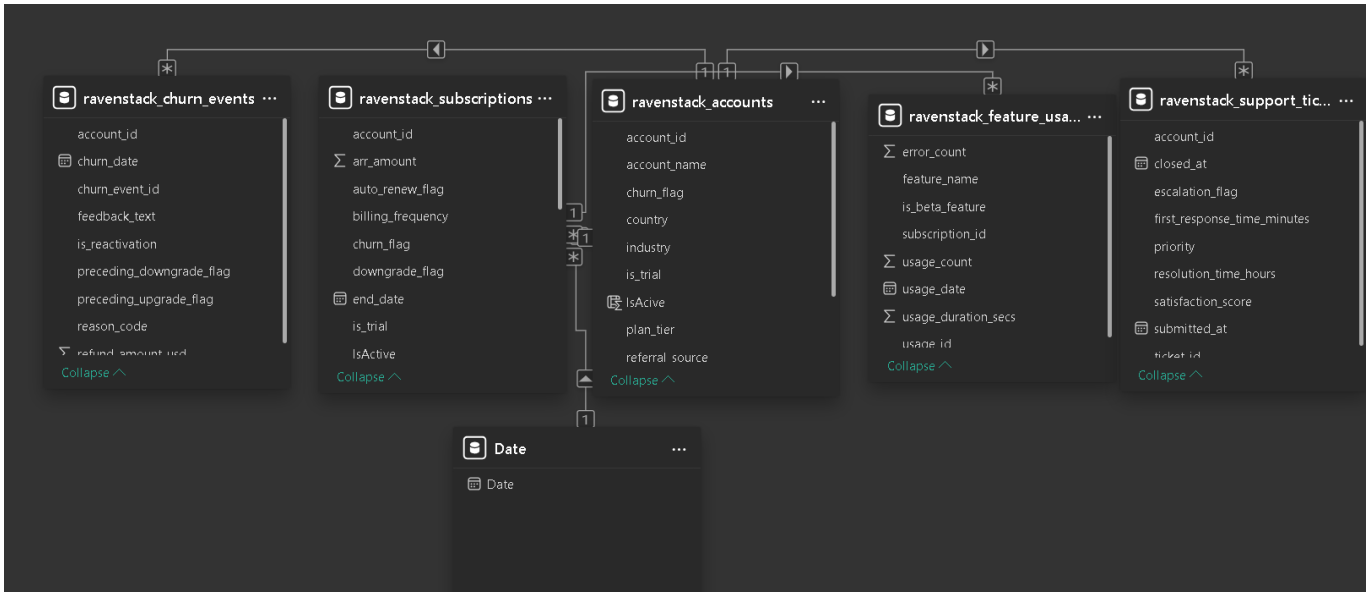


8.2 RavenStack SaaS Dataset:

Dashboard View:

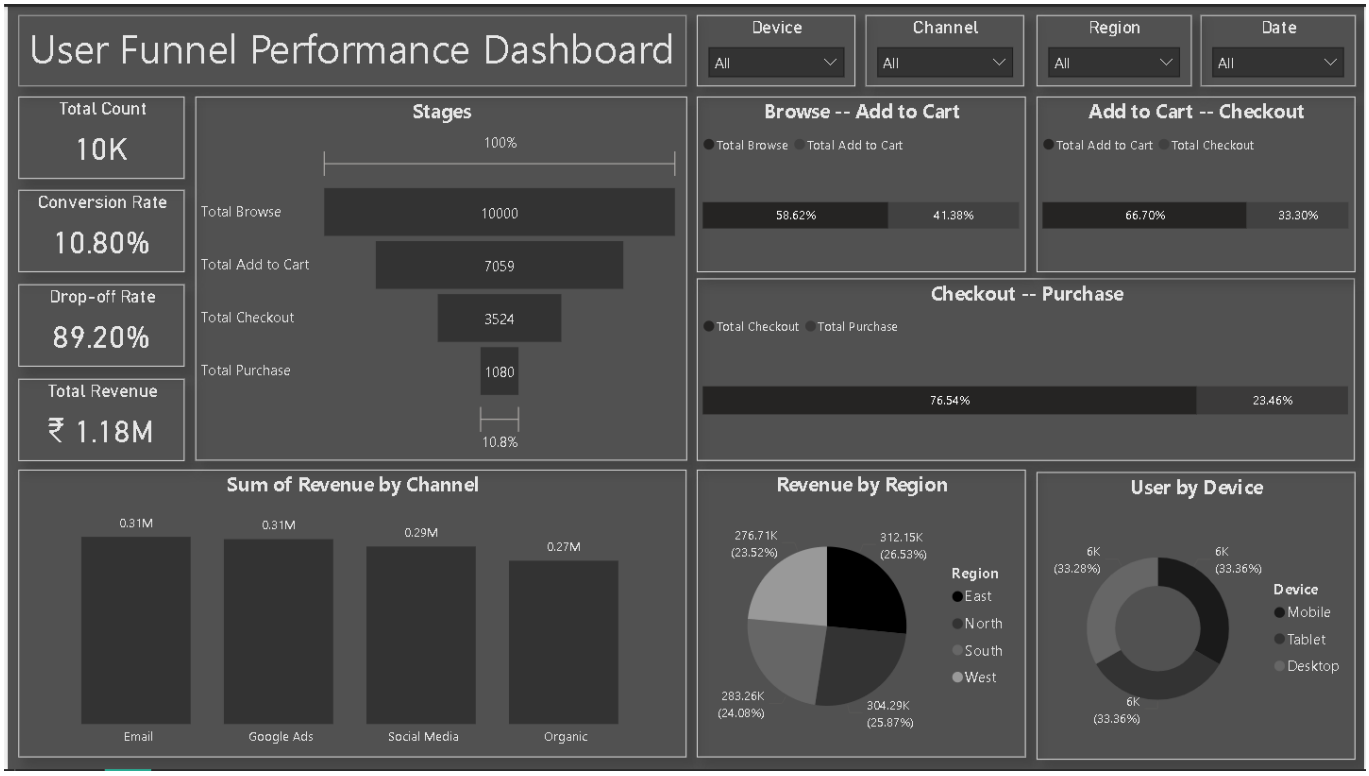


Model View:

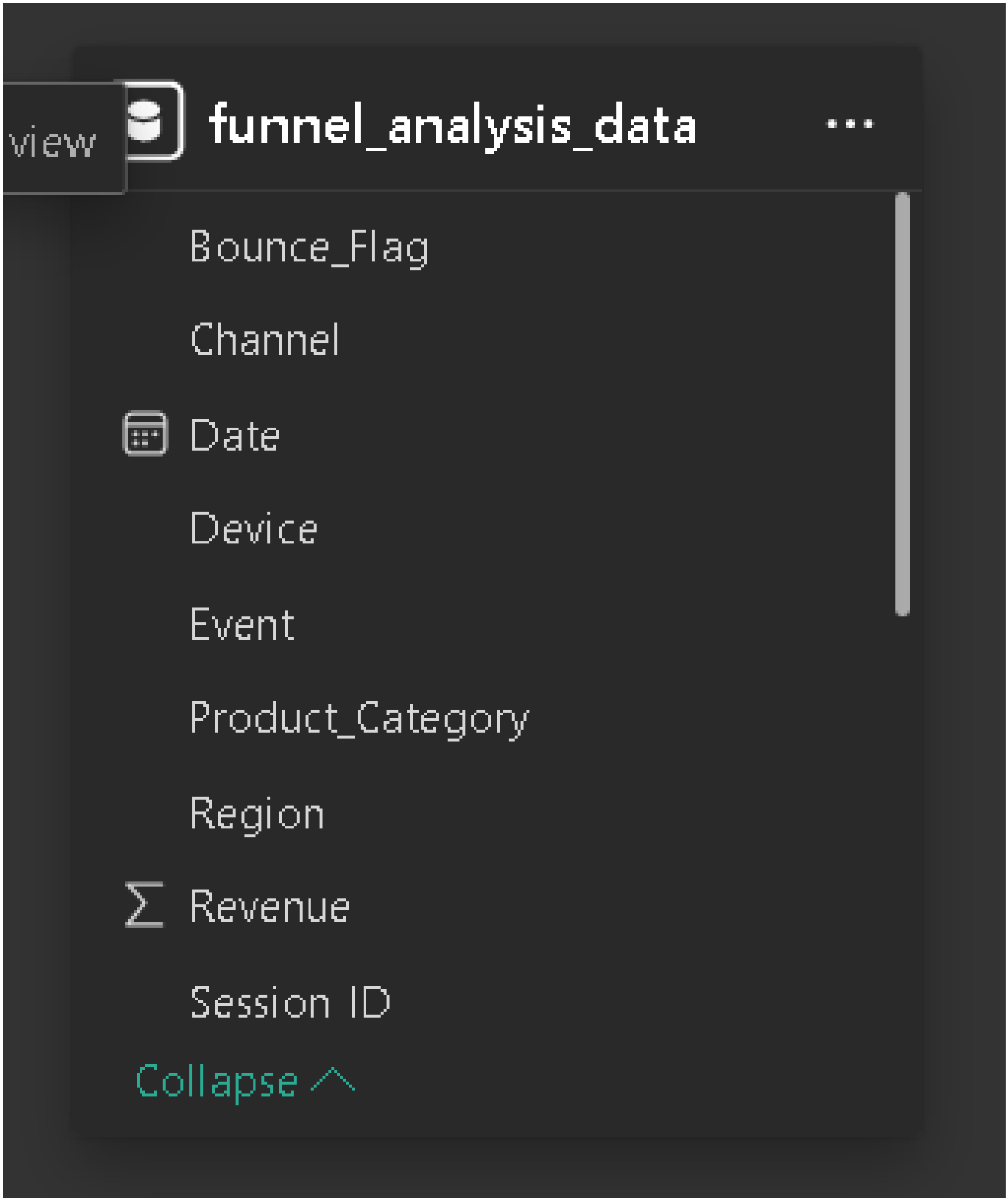


8.3 E-Commerce Funnel Analysis:

Dashboard View:



Model View:


view
funnel_analysis_data
Bounce_Flag
Channel
Date
Device
Event
Product_Category
Region
Revenue
Session ID
Collapse

Chapter 9: Limitation and Future Enhancements

9.1 Limitation

* Dashboards rely on static file updates (CSV/Excel) rather than live database connections.

* Current models lack predictive machine learning algorithms (e.g., predicting exact churn dates).

9.2 Future Enhancement

* Integration of a SQL database (like PostgreSQL) for scalable, live data storage.

* Automating the execution of Python scripts on a weekly schedule.

* Implementing A/B testing analysis and churn prediction models based on user behavior patterns.

Chapter 10: Conclusion

The completed Data Analytics Internship tasks successfully established robust data extraction, cleaning, and visualization pipelines. By migrating raw, disjointed data into structured, interactive environments (Excel, Power BI), the projects deliver clear visibility into business performance, user journeys, and customer retention metrics. The solutions designed are scalable and provide an excellent foundation for more advanced predictive analytics in the future.